AI Assisted Coding
Lab Assignment - 20.5

Name : Likhith Sai
Hall.t.No. : 2303A51026
Batch No. : 01

Lab 20 – Security Testing: Identifying Vulnerabilities in AI- Generated Code

Lab Objectives:

- Understand how to test AI-generated code for common security vulnerabilities.
- Learn to apply secure coding principles while analyzing AI outputs.
- Practice detecting risks such as SQL injection, XSS, hardcoded credentials, and weak encryption.
- Enhance code reliability and safety by using AI for secure refactoring.

Task 1 – Password Storage Vulnerability Check

Task: Analyze an AI-generated signup script for insecure password storage.

Instructions:

- Prompt AI to generate a Python user registration program.
- Check whether passwords are stored in plain text.
- Improve the script using hashing (bcrypt or hashlib).

Expected Output:

- A secure registration script with hashed password storage.

Copilot Suggested Code :

```python
# Task: Analyze an AI-generated signup script for insecure password storage.

# AI-generated user registration program
def register_user(username, password):
    # Insecure password storage (plain text)
    with open('users.txt', 'a') as file:
        file.write(f'{username}:{password}\n')
    print("User registered successfully!")
# Example usage
register_user('john_doe', 'password123')
# Improved registration script with hashed password storage
import bcrypt
def register_user_secure(username, password):
    # Hash the password using bcrypt
    hashed_password = bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt())
    with open('users_secure.txt', 'a') as file:
        file.write(f'{username}:{hashed_password.decode("utf-8")}\n')
    print("User registered securely!")
# Example usage
register_user_secure('john_doe', 'password123')

# Analysis:
# The original registration script stores passwords in plain text, which is a significant security risk. If the 'users.txt' file is compromised, attackers c
# The improved registration script uses bcrypt to hash passwords before storing them. This means that even if the 'users_secure.txt' file is compromised, at
```

Output :

```
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-20.5> & "C:\Program Files\Python314\python.exe" c:/B.Tech/3-2/AI_Assisted_Coding/lab_assignment-20.5/task_1.py
User registered successfully!
User registered securely!
```

Task 2 – Hardcoded Secrets Detection

Task: Evaluate AI-generated code for hardcoded API keys or credentials.

Instructions:

- Ask AI to generate a script that connects to an external API.
- Identify if API keys/passwords are written directly in code.
- Refactor using environment variables or config files.

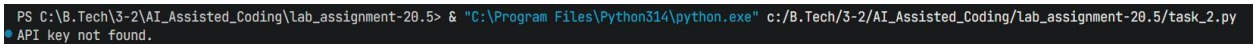
Expected Output:

- A secure script with secrets managed properly.

Copilot Suggested Code :

```python
# Task: Evaluate AI-generated code for hardcoded API keys or credentials.

# AI-generated script with hardcoded API key
import requests
import os

def fetch_data_secure():
    api_key = os.getenv('API_KEY')

    if not api_key:
        print("API key not found.")
        return

    url = f'https://jsonplaceholder.typicode.com/posts'

    try:
        response = requests.get(url, timeout=5)
        response.raise_for_status()
        print("Data fetched successfully!")
        print(response.json()[:2])  # sample output

    except requests.exceptions.RequestException as e:
        print("Error:", e)

fetch_data_secure()
# Analysis:
# The original script contains a hardcoded API key, which is a security risk. If the code is shared or exposed, the API key can be easily extracted and misu
# The refactored script retrieves the API key from an environment variable, which is a more secure practice. This way, the API key is not exposed in the coc
```

Output :



```
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-20.5> & "C:\Program Files\Python314\python.exe" c:/B.Tech/3-2/AI_Assisted_Coding/lab_assignment-20.5/task_2.py
API key not found.
```

Task 3 – File Upload Security Validation

Task: Test an AI-generated file upload program for unsafe file handling.

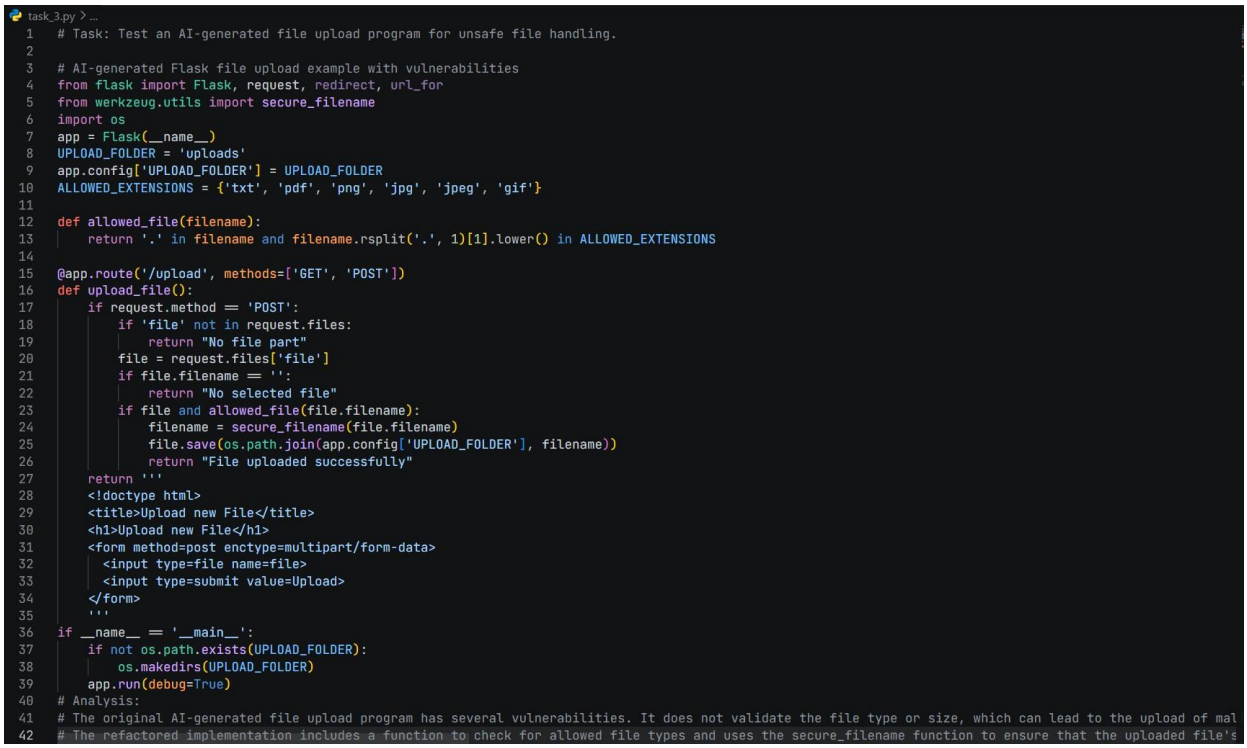
Instructions:

- Prompt AI to generate a Flask file upload example.
- Check for vulnerabilities like uploading executable files.
- Add validation for file type, size, and secure filename Handling.

Expected Output:

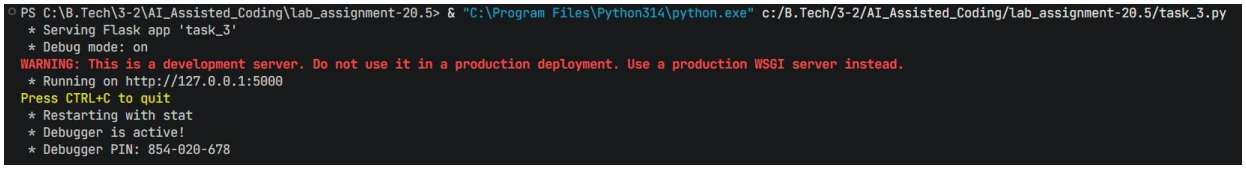
- A secure file upload implementation.

Copilot Suggested Code :



```python
# Task: Test an AI-generated file upload program for unsafe file handling.

# AI-generated Flask file upload example with vulnerabilities
from flask import Flask, request, redirect, url_for
from werkzeug.utils import secure_filename
import os
app = Flask(__name__)
UPLOAD_FOLDER = 'uploads'
app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER
ALLOWED_EXTENSIONS = {'txt', 'pdf', 'png', 'jpg', 'jpeg', 'gif'}

def allowed_file(filename):
    return '.' in filename and filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS

@app.route('/upload', methods=['GET', 'POST'])
def upload_file():
    if request.method == 'POST':
        if 'file' not in request.files:
            return "No file part"
        file = request.files['file']
        if file.filename == '':
            return "No selected file"
        if file and allowed_file(file.filename):
            filename = secure_filename(file.filename)
            file.save(os.path.join(app.config['UPLOAD_FOLDER'], filename))
            return "File uploaded successfully"
    return '''
    <!doctype html>
    <title>Upload new File</title>
    <h1>Upload new File</h1>
    <form method=post enctype=multipart/form-data>
      <input type=file name=file>
      <input type=submit value=Upload>
    </form>
    '''
if __name__ == '__main__':
    if not os.path.exists(UPLOAD_FOLDER):
        os.makedirs(UPLOAD_FOLDER)
    app.run(debug=True)
# Analysis:
# The original AI-generated file upload program has several vulnerabilities. It does not validate the file type or size, which can lead to the upload of mal
# The refactored implementation includes a function to check for allowed file types and uses the secure_filename function to ensure that the uploaded file's
```

Output :



```
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-20.5> & "C:\Program Files\Python314\python.exe" c:/B.Tech/3-2/AI_Assisted_Coding/lab_assignment-20.5/task_3.py
 * Serving Flask app 'task_3'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 854-020-678
```

Task 4 – Command Injection Prevention

Task: Analyze an AI-generated Python script that executes system commands.

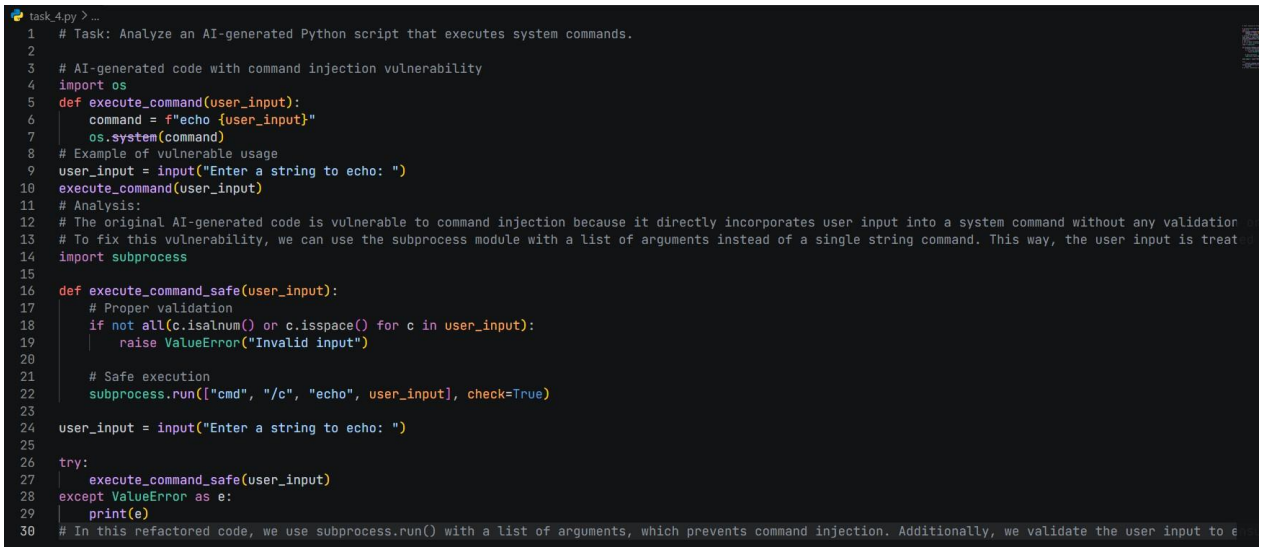
Instructions:

- Ask AI to generate code using os.system() or subprocess.
- Identify command injection risks with user input.
- Fix using safe subprocess calls and input validation.

Expected Output:

- A secure command execution script.

Copilot Suggested Code :



```python
# Task: Analyze an AI-generated Python script that executes system commands.

# AI-generated code with command injection vulnerability
import os
def execute_command(user_input):
    command = f"echo {user_input}"
    os.system(command)
# Example of vulnerable usage
user_input = input("Enter a string to echo: ")
execute_command(user_input)
# Analysis:
# The original AI-generated code is vulnerable to command injection because it directly incorporates user input into a system command without any validation
# To fix this vulnerability, we can use the subprocess module with a list of arguments instead of a single string command. This way, the user input is treat
import subprocess

def execute_command_safe(user_input):
    # Proper validation
    if not all(c.isalnum() or c.isspace() for c in user_input):
        raise ValueError("Invalid input")

    # Safe execution
    subprocess.run(["cmd", "/c", "echo", user_input], check=True)

user_input = input("Enter a string to echo: ")

try:
    execute_command_safe(user_input)
except ValueError as e:
    print(e)
# In this refactored code, we use subprocess.run() with a list of arguments, which prevents command injection. Additionally, we validate the user input to e
```

Output :

```
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-20.5> & "C:\Program Files\Python314\python.exe" c:/B.Tech/3-2/AI_Assisted_Coding/lab_assignment-20.5/task_4.py
Enter a string to echo: hello
hello
Enter a string to echo: mendis
mendis
```

Task 5 – Secure Input Handling in Web Forms

Task: Check AI-generated web form code for improper input validation.

Instructions:

- Ask AI to generate an HTML + Flask feedback form.
- Identify missing validation and unsafe rendering.
- Fix using server-side validation and sanitization.

Expected Output:

- A secure form resistant to injection attacks.

Copilot Suggested Code :

```
task_5.py > ...
# Task: Check AI-generated web form code for improper input validation.

# AI-generated code for a feedback form using Flask
from flask import Flask, request, render_template_string
app = Flask(__name__)
@app.route('/feedback', methods=['GET', 'POST'])
def feedback():
    if request.method == 'POST':
        user_feedback = request.form['feedback']
        # Unsafe rendering of user input
        return render_template_string(f"<h1>Your Feedback: {user_feedback}</h1>")
    return '''
        <form method="post">
            <textarea name="feedback"></textarea>
            <input type="submit" value="Submit">
        </form>
    '''
if __name__ == '__main__':
    app.run(debug=True)
# Analysis:
# The original AI-generated code is vulnerable to Cross-Site Scripting (XSS) attacks because it directly renders user input without any sanitization. An att
# To fix this vulnerability, we can use the `escape` function from the `markupsafe` module to sanitize the user input before rendering it. This will ensure
from flask import Flask, request, render_template_string
from markupsafe import escape
app = Flask(__name__)
@app.route('/feedback', methods=['GET', 'POST'])
def feedback():
    if request.method == 'POST':
        user_feedback = request.form['feedback']
        # Safe rendering of user input
        safe_feedback = escape(user_feedback)
        return render_template_string(f"<h1>Your Feedback: {safe_feedback}</h1>")
    return '''
        <form method="post">
            <textarea name="feedback"></textarea>
            <input type="submit" value="Submit">
        </form>
    '''
if __name__ == '__main__':
    app.run(debug=True)
# In this refactored code, we use the `escape` function to sanitize the user input before rendering it. This prevents any malicious code from being executed
```

Output :

```
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-20.5> & "C:\Program Files\Python314\python.exe" c:/B.Tech/3-2/AI_Assisted_Coding/lab_assignment-20.5/task_5.py
 * Serving Flask app 'task_5'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 854-020-678
 * Detected change in 'c:\\B.Tech\\3-2\\AI_Assisted_Coding\\lab_assignment-20.5\\task_5.py', reloading
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 854-020-678
 * Detected change in 'c:\\B.Tech\\3-2\\AI_Assisted_Coding\\lab_assignment-20.5\\task_5.py', reloading
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 854-020-678
 * Detected change in 'c:\\B.Tech\\3-2\\AI_Assisted_Coding\\lab_assignment-20.5\\task_5.py', reloading
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 854-020-678
 * Detected change in 'c:\\B.Tech\\3-2\\AI_Assisted_Coding\\lab_assignment-20.5\\task_5.py', reloading
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 854-020-678
```